

Support of Sparse Tensor Computing for MLIR HLS

Geng-Ming Liang
Department of Computer Science,
National Tsing Hua University
Hsinchu, Taiwan
gmliang@pllab.cs.nthu.edu.tw

Chao-Lin Lee
Department of Computer Science,
National Tsing Hua University
Hsinchu, Taiwan
clli@pllab.cs.nthu.edu.tw

Robert Lai
Mediatek Inc.
Hsinchu, Taiwan
Robert.Lai@mediatek.com

Jenq-Kuen Lee
Department of Computer Science,
National Tsing Hua University
Hsinchu, Taiwan
jklee@cs.nthu.edu.tw

ABSTRACT

Nowadays, sparse tensor computations are widely used in machine learning. Without the multiplications in zero values, sparse tensor computation can significantly reduce the latency and power consumption. Famous frameworks like TensorFlow, PyTorch, Pandas, etc., already have the support of sparse tensor computing. MLIR also integrated this idea and implemented the compilation flow. Integrating sparse tensor computing and MLIR into High-level Synthesis (HLS) can generate more powerful RTL and further implement specified hardware. However, MLIR flow isn't well done now while translating into LLVM IR and does not fully support HLS tools, which are not supporting MLIR. In this paper, we propose a flow in MLIR to lower sparse tensor computations into HLS-readable LLVM IR, which can then be synthesized into RTL. To demonstrate the effectiveness of our proposed flow, we devise experiments by implementing matrix multiplication operations in the convolution layers, and the data is pruned to maximize the sparsity. Our proposed flow speeds up the latency by about 3.6 times, as demonstrated in our experiment with Xilinx Vitis HLS.

CCS CONCEPTS

• **Software and its engineering** → **Source code generation**; **Parsers**.

KEYWORDS

MLIR, HLS, LLVM, Sparse-Tensor

ACM Reference Format:
Geng-Ming Liang, Chao-Lin Lee, Robert Lai, and Jenq-Kuen Lee. 2023. Support of Sparse Tensor Computing for MLIR HLS. In *52nd International Conference on Parallel Processing Workshops (ICPP-W 2023), August 07–10, 2023, Salt Lake City, UT, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3605731.3605908

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICPP-W 2023, August 07–10, 2023, Salt Lake City, UT, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0842-8/23/08…$15.00
https://doi.org/10.1145/3605731.3605908

ACKNOWLEDGMENTS

This work is supported in part by Taiwan NSTC and Mediatek.

1 INTRODUCTION

Machine learning has become ubiquitous worldwide, with many technologies incorporating it to enhance their services, especially on edge devices. Edge devices, such as smartphones and IoT devices, have limited computing resources. Consequently, optimizations in tensor computing, the most important part of machine learning, are essential for researchers. Sparse tensor computing is a classic and efficient approach to accelerate tensor computations. And it is largely used in image-related classifications. By eliminating redundant zero computations, we can significantly reduce computation time, with higher sparsity (more zeros in the tensor) leading to greater time savings. Notably, sparse tensor computing is already supported by popular AI frameworks like PyTorch, TensorFlow, and so on. In recent years, Multi-level Intermediate Representation (MLIR) [9] has integrated sparse tensor capabilities and provided a compilation flow. MLIR offers a convenient interface for various front-ends and enables optimizations within and across different abstraction levels. Leveraging MLIR and sparse tensor computing in High-Level Synthesis (HLS) allows us to generate more efficient Register Transfer Level (RTL) designs for specialized AI hardware circuits. However, the sparse tensor compilation flow provided by MLIR is not fully implemented, and MLIR lacks support for current HLS tools based on older versions of LLVM.

In this paper, we present a comprehensive compilation flow that translates MLIR to LLVM IR and further synthesizes it into RTL using HLS tools. The current version of MLIR lacks the implementation of sparse pointers/indices/values functions, and the multi-dimensional array lowering is not compatible with the older version of LLVM on which HLS tools rely. Additionally, the syntax of the newer LLVM version differs slightly from the older version, and HLS tools require directive information for RTL synthesis. With our adaptor, we bridge the gap between unimplemented features of MLIR and unsupported syntax between different LLVM versions. This enables developers to easily generate sparse tensor RTL using MLIR and HLS tools, allowing them to focus on other optimizations such as data pruning. Experimental results using MNIST data on Xilinx Vitis HLS demonstrate that leveraging sparse tensor computing can reduce latency by approximately 3.6 times.

The remainder of this paper is organized as follows. In Section 2, we will describe the background of sparse matrices, MLIR, and HLS. The problem detail of the current MLIR and the sparse compiler will be represented in Section 3. The detailed flow of our work is in Section 4. Section 5 shows our experiment results, and we will give a conclusion in Section 6 to conclude this paper.

2 BACKGROUND

2.1 Sparse Matrices

Sparse matrices, where the majority of elements are zero, are widely used in various domains, including scientific computing, graph theory, machine learning, and data analysis. Efficient representations of sparse matrices can significantly impact the performance of operations and algorithms. In this section, we introduce the background of sparse matrices and discuss different representation formats, including Coordinate (COO), Compressed Sparse Row (CSR), and Block Sparse Row (BSR). We also introduce pruning techniques to increase sparsity and provide a brief overview of relevant papers.

Two key properties of sparse matrices are:

- Sparsity: A high proportion of zero elements compared to non-zero elements.
- Storage efficiency: Efficient storage techniques minimize memory usage.

Several formats have been proposed for representing sparse matrices, focusing on optimizing storage and computational efficiency. The Coordinate (COO) format represents a sparse matrix using three arrays: row indices, column indices, and non-zero values. Each non-zero element in the matrix is stored as a triplet (row, column, value). The Compressed Sparse Row (CSR) format compresses the row information to represent a sparse matrix. It uses three arrays: non-zero values, column indices, and row pointers. CSR format enables efficient row-wise operations, including matrix-vector multiplication, and has been utilized in research such as the Halide scheduling primitive for CSR and hybrid compression methods [10]. The Block Sparse Row (BSR) format represents a sparse matrix by dividing it into smaller fixed-size dense blocks. It uses four arrays: non-zero block values, block column indices, row pointers, and a block size parameter.

Pruning techniques are used to increase the sparsity in neural networks and other data structures by removing less important connections or elements. These techniques can lead to reduced memory consumption and computational requirements. There are several relevant researches that utilize pruning to enhance sparsity. The study by [5] integrates pruning, quantization, and Huffman coding to compress deep neural networks without sacrificing their accuracy. On the other hand, [4] suggests that there exist successful subnetworks within large neural networks that can be discovered through pruning. Additionally, the research regarding the convolution of CNN with compressed sparse matrix multiplication flow in TVM is presented in [12], which applies distiller for weight pruning and pretraining and carries out CSR and BSR sparse matrix multiplications in TVM. Finally, [3] introduces a heuristic algorithm that automatically selects compression and distribution schemes for sparse data arrays.

2.2 MLIR

Although LLVM provides a unified interface that allows various high-level languages to parse their syntax to LLVM IR and further to the target machine IR, such as Clang AST, Java BC, and so on. Higher-level abstraction languages often generate their own IR similar to LLVM IR with domain-specific optimization, library-specific optimization, and lots of other things. However, this can lead to redundant processes being performed by various front-ends before entering LLVM, resulting in repetitive code writing. To address this problem, MLIR was introduced as a sub-project of LLVM. MLIR shares many features with LLVM and follows the SSA-Based compilation strategy, employing a Module/Function/Block/Operation structure. One important aspect of MLIR is "**Dialect**", which defines a namespace for a group of related attributes, types, operations, and transformations. Figure 1 shows an example of MLIR syntax. The red text, starting with '%', represents registers, that can store the result of an operation or be an operand in operations. The purple text denotes the dialect name, while the blue text represents the operation specific to that dialect. The pink text describes the attribute value, and the orange text represents the type within the dialect. There are a few standard dialects that MLIR provides. We

```
%result = dialect0.operation(%op0, %op1)
    {attribute = dialect1<"value">}
    : (!dialect.type) -> !dialect.resultType
```

Figure 1: An example of an MLIR operation. The `%result` register stores the value from the operation in `dialect0`. This operation has the attribute from `dialect1`. The type of operation and the type of the result is after the ":" character. Each color represents dialect, type, attribute value, operation, and register.

list a few dialects here;

- `Affine`: Providing abstraction of affine operations (e.g. `affine.for`, `affine.if`) and further making the dependence analysis and loop transformations.
- `SCF`: SCF stands for structured control flow. Defines high-level control flow operations (e.g. `scf.for`, `scf.if`). Different from `Affine` dialect, `SCF` can use any SSA values as loop bounds and conditions, and build more flexible programs.

Benefiting from the multi-level abstraction, developers can focus on their domain-specific features and optimizations, then further convert their own dialects into standard dialects. Without rewriting similar implementations, the development time can be significantly reduced. Famous AI frameworks like PyTorch, and ONNX, have already implemented their own dialect in MLIR to represent graph-level abstraction.

2.3 HLS

With the growth of chip designs becoming more and more complex, traditional RTL designs in a whole system will cause more engineering costs and be hard to debug and maintain. High-level synthesis(HLS) tools help researchers significantly reduce development time. Without knowing too many hardware design details,

directives in HLS tools help developers describe the hardware behaviors. HLS tools, or HLS C/C++, are usually based on C/C++ syntax, and use an important part, pragma, to design the synthesis policy. Pragmas can not only direct how the code is optimized but also determine the hardware usage. Listing 1 shows an example using Xilinx Vitis HLS. It describes the simple code of matrix multiplication `C[i,j] = A[i,k] * B[k,j]`. Based on the C/C++ syntax, Lines 2 and 6 add two pragmas. Pragma `top` describes the name of the top function, and pragma `unroll` describes the for loop that should be fully unrolled.

```
1  void MxM(int A[3][4], int B[4][2], int C[3][2]) {
2  #pragma HLS top name=MxM
3      for (int i = 0; i < 3; i++) {
4          for (int j = 0; j < 2; j++) {
5              for (int k = 0; k < 4; k++) {
6  #pragma HLS unroll
7                  C[i][j] += A[i][k] * B[k][j];
8              }
9          }
10     }
11 }
```

Listing 1: An example of `C[i,j] = A[i,k] * B[k,j]` in Xilinx Vitis HLS. 2 pragmas are used in Lines 2 and 6. The first pragma `top` tells the HLS the top function is *MxM*. `unroll` directive describes this matrix multiplication needs to be fully unrolled.

3 THE PROBLEM OF MLIR & SPARSE

3.1 Multi-dimension Arrays in MLIR

In machine learning, the most important part is tensor computations, like convolution layers, pooling layers, etc. The later release of LLVM(15 or later) deals with the tensor arguments into opaque pointers. That is to say while loading or storing values into tensors, LLVM determines the access size to calculate the right position to access, and so is its sub-project MLIR. However, this feature is on the opposite side of HLS tools. In HLS tools, they need storage information to estimate how many storage spaces are needed and further use the optimization directives, like array partition and so on. Since we want to utilize the MLIR spares-tensor compiler flow to generate more powerful RTLs, we decide to fix this feature in `memref` to `llvm` dialects conversion in MLIR. We will explain more details in Section 4.2.

3.2 Sparse-tensor Dialect Syntax

With the sparse tensor computing arising in a lot of scenarios, MLIR decides to integrate this idea and provide a compiler flow [1]. In this section, we will introduce the features and syntaxes of MLIR `SparseTensor` dialect. Based on the basic MLIR dialects, `SparseTensor` dialect is designed to be a first-class citizen and provides the attributes, types, operations, and transformations. Without changing the original tensor computations, `SparseTensor` dialect only adds an attribute in `Tensor` dialect to provide additional information about the compress patterns. Listing 2 shows an example of `SparseTensor` dialect attribute syntax. Lines 1-5 define the encoding information. The *dimLevelType* describes whether each dimension is compressed or not. Lines 6 and 7 describe the matrix multiplication `C[i,j] += A[i,k] * B[k,j]` in dense and sparse ways, separately. As you can see, the integration almost keeps the original syntax, only adding the attribute `CSR` in the `Tensor` dialect. Other operations like `sparse_tensor.convert`, `sparse_tensor.new`, and so on, are used to create the sparse tensor to match the type in `linalg` dialect computations.

```
1 #CSR = #sparse_tensor.encoding<{
2     dimLevelType = [ "dense", "compressed"],
3     dimOrdering = affine_map<(i,j) -> (i,j)>,
4     pointerBitWidth = ...,
5     indexBitWidth = ..., }>
6 %C = linalg.matmul ins(%A,%B: tensor<?x?xf64>,tensor<?x?
      xf64>) -> tensor<?x?xf64>
7 %D = linalg.matmul ins(%A,%B: tensor<?x?xf64,#CSR>,tensor
      <?x?xf64,#CSR>) -> tensor<?x?xf64,#CSR>
```

Listing 2: An example of `SparseTensor` encoding and matrix multiplication in `Linalg`, which are represented `C[i,j] = A[i,k] * B[k,j]` in both dense and sparse way.

For the transformation in `SparseTensor` dialect, it follows the *sparse iteration model* of TACO to lower to the `SCF` dialect. With the pass *sparsification*, sparse tensor computations change into different `SCF` loop iterations depending on the compressed format. Listing 3 shows the result of **CSR** format. As you can see Lines 18 and 20 calculate the start/end pointer to the indices and values arrays. You can see there are 3 functions generated, `SparseValues`, `SparseIndices`, and `SparsePointers`. All these 3 functions pass the pointer in *i8* type to describe the opaque pointer, as we mentioned above, and their return types are all *unrankedmemref* type, this is called the *one-size-fits-all* solution. MLIR also specifies the **COO** version, but we won't talk about that in this paper.

```
1  module {
2    func.func private @sparseValuesF64(!llvm.ptr<i8>) ->
       memref<?xf64> attributes {llvm.emit_c_interface}
3    func.func private @sparseIndices0(!llvm.ptr<i8>, index)
       -> memref<?xindex> attributes {
       llvm.emit_c_interface}
4    func.func private @sparsePointers0(!llvm.ptr<i8>, index
       ) -> memref<?xindex> attributes {
       llvm.emit_c_interface}
5    func.func @spMxM(%arg0: !llvm.ptr<i8>, %arg1: tensor<4
       x2xf64>, %arg2: tensor<3x2xf64>) -> tensor<3x2xf64>
       {
6      %c3 = arith.constant 3 : index
7      %c2 = arith.constant 2 : index
8      %c0 = arith.constant 0 : index
9      %c1 = arith.constant 1 : index
10     %c1_0 = arith.constant 1 : index
11     %0 = call @sparsePointers0(%arg0, %c1_0) : (!llvm.ptr
       <i8>, index) -> memref<?xindex>
12     %c1_1 = arith.constant 1 : index
13     %1 = call @sparseIndices0(%arg0, %c1_1) : (!llvm.ptr<
       i8>, index) -> memref<?xindex>
14     %3 = bufferization.to_memref %arg1 : memref<4x2xf64>
15     %4 = bufferization.to_memref %arg2 : memref<3x2xf64>
16     scf.for %arg3 = %c0 to %c3 step %c1 {
17       %6 = memref.load %0[%arg3] : memref<?xindex>
18       %7 = arith.addi %arg3, %c1 : index
19       %8 = memref.load %0[%7] : memref<?xindex>
20       scf.for %arg4 = %6 to %8 step %c1 {
21         %9 = memref.load %1[%arg4] : memref<?xindex>
22         %10 = memref.load %2[%arg4] : memref<?xf64>
23         scf.for %arg5 = %c0 to %c2 step %c1 {
24           %11 = memref.load %4[%arg3, %arg5] : memref<3
       x2xf64>
```

```
25          %12 = memref.load %3[%9, %arg5] : memref<4
    x2xf64>
26          %13 = arith.mulf %10, %12 : f64
27          %14 = arith.addf %11, %13 : f64
28          memref.store %14, %4[%arg3, %arg5] : memref<3
    x2xf64>
29        }
30      }
31    }
32    %5 = bufferization.to_tensor %4 : memref<3x2xf64>
33    return %5 : tensor<3x2xf64>
34  }
35 }
```

Listing 3: An example shows the matrix multiplication `C[3,2] = A[3,4] * B[4,2]`. The first argument `%arg0` is compressed in CSR format, others are dense tensors. Lines 11-13 call 3 sparse functions to return pointers, indices, and values, respectively. Lines 18 and 20 get the pointers. Line 22 gets the indices and Line 23 gets the values. This MLIR is after using *sparsification* and *sparse-tensor-conversion* passes.

4 FRAMEWORK DESIGN

4.1 Overview of the Framework Design

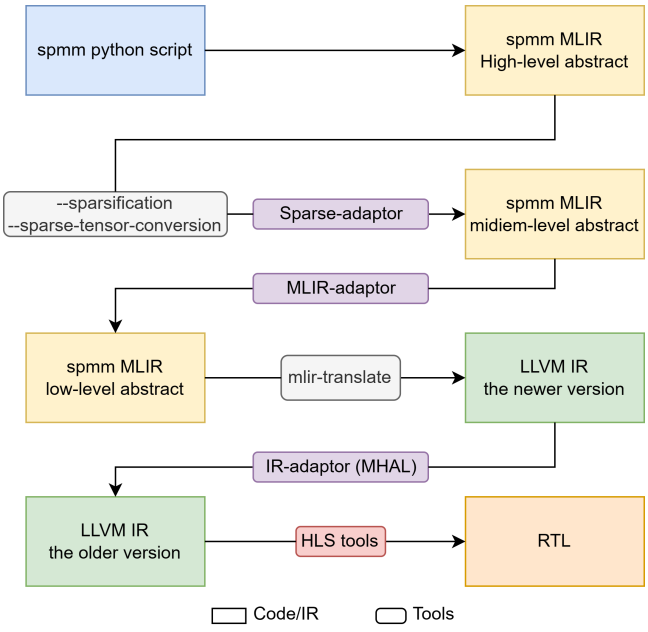


Figure 2: The framework of our adaptor is presented. Three adaptors are used in the framework dealing with sparse features, MLIR syntax, and LLVM version lowering respectively. We believe that the Python script can be adapted to various high-level languages. Our adaptor addresses the issues with features that are not well-implemented by assisting in fixing the syntax during dialect lowering and LLVM IR version lowering.

With the integration of sparse tensor computing in MLIR, we can generate more powerful IR from various frontend languages. Furthermore, utilizing the LLVM IR, which is translated from MLIR can make HLS tools available to more variety of high-level languages and benefit from optimizations from MLIR. However, the compilation flow of sparse tensor dialect in MLIR is not well-implemented and current HLS tools, which are based on the older version of LLVM, are not supporting MLIR syntax. Take Xilinx Vitis HLS as an example, it is based on LLVM 7, which is not supported for MLIR (since LLVM 10), and the syntax of LLVM differs between various versions. To address these problems, we propose an adaptor to complete the compilation flow to support sparse tensor computation from MLIR to HLS tools. Figure 2 shows the framework of our work. Three adaptors involved in the framework help the syntax correctly be fixed. Sparse-adaptor dealing with the sparse related features, such as sparse functions and sparse arrays. MLIR-adaptor solves the unsupported syntax while lowering dialects in MLIR. IR-adaptor is our previous work MHAL [11] for LLVM version lowering. For the frontend part, current MLIR only supports Python to sparse tensor compilation flow, but we believe that in the future, there will be lots of high-level languages that can compile into `SparseTensor` dialect. With the help of our adaptor, various tensor computing, which are using sparse optimization, can be integrated into HLS tools and generate more powerful hardware circuits.

4.2 Adaptor Design

To complete the compilation flow of sparse tensor computing in MLIR, we need to know every detail of IR in each step.

4.2.1 Sparse-adaptor. For the first step, we generate the highest abstract level of MLIR with `SparseTensor` dialect from Python script as shown in Listing 4. As you can see, `%arg1` contains sparse information and MLIR using `Linalg` dialect to calculate the matrix multiplication.

```
1  #map0 = affine_map<(d0, d1, d2) -> (d1, d0)>
2  #map1 = affine_map<(d0, d1, d2) -> (d0, d2)>
3  #map2 = affine_map<(d0, d1, d2) -> (d1, d2)>
4  #CSR = #sparse_tensor.encoding<{
5      dimLevelType = [ "dense", "compressed" ],
6      dimOrdering = affine_map<(d0, d1) -> (d0, d1)>
7  }>
8  module {
9    func.func @spMxM(%arg0: tensor<3x4xf64>, %arg1: tensor
       <4x2xf64, #CSR>, %arg2: tensor<3x2xf64>) -> tensor<3
       x2xf64> {
10     %0 = linalg.generic {indexing_maps = [#map0, #map1, #
       map2], iterator_types = ["reduction", "parallel", "
       parallel"]}
11     ins(%arg0, %arg1 : tensor<3x4xf64>, tensor<4x2xf64, #
       CSR>) outs(%arg2 : tensor<3x2xf64>) {
12     ^bb0(%arg3: f64, %arg4: f64, %arg5: f64):
13       %1 = arith.mulf %arg3, %arg4 : f64
14       %2 = arith.addf %arg5, %1 : f64
15       linalg.yield %2 : f64
16     } -> tensor<3x2xf64>
17     return %0 : tensor<3x2xf64>
18   }
19 }
```

Listing 4: Showing `C[3,2] = A[3,4] * B[4,2]`, and B is sparse matrix in CSR format. The matrix multiplication is calculated in `Linalg` operation in Lines 10-16.

4.2.2 MLIR-adaptor. In the next step, we utilize the "sparsification" pass to lower the SparseTensor dialect into standard dialects, as described in Section 3.2. Due to the lack of support for opaque pointers in the older version of LLVM on which HLS tools are based, we convert the type to the memref dialect for both "SparseTensor" and other "tensor" dialects. Additionally, HLS tools typically expect functions to have void return types. To address this, we modify all functions, including the main function (spMxM), and all sparse functions, to have void return types. Since we need three spaces to store information in sparse functions, we create three arrays using memref.alloc() to allocate them respectively, as shown in Lines 1-3 of Listing 5. Furthermore, we implement the *sparseValues*, *sparseIndices*, and *sparsePointers* functions using different compression patterns based on the attributes of the "SparseTensor". Currently, we only implement the CSR (Compressed Sparse Row) compression format. More compression schemes are being added to our research work.

```
1 %sp = memref.alloca() : memref<4xi64>
2 %si = memref.alloca() : memref<12xi64>
3 %sv = memref.alloca() : memref<12xf64>
4 call @sparsePointers(%arg0, %sp) : (memref<3x4xf64>,
      memref<4xi64>) -> ()
5 call @sparseIndices(%arg0, %si) : (memref<3x4xf64>,
      memref<12xi64>) -> ()
6 call @sparseValues(%arg0, %sv) : (memref<3x4xf64>, memref
      <12xf64>) -> ()
```

Listing 5: We allocate 3 arrays to store the sparse pointers, indices, and values respectively. We allocate 3 arrays to store the sparse pointers, indices, and values respectively. Subsequently, we can invoke the functions we have implemented based on different compression patterns. As the return types of these functions are all void, they are represented as "()" in the corresponding operations, as shown in Lines 4-6.

After the first fixing, we lower SCF dialect to a lower level dialect, CF, and other dialects, except memref and func dialects, are lowered into llvm dialect. The reason that we keep memref and func dialect is that, the current MLIR lowering feature of these 2 dialects is not fit in the older version of LLVM while the argument is multi-dimension arrays. Listing 6 shows the result of using *convert-memref-to-llvm* and *convert-func-to-llvm="use-bare-ptr-memref-call-conv"* passes. All function arguments are changed into bare pointers, and the access pattern is not suitable for the older version of LLVM. And a similar problem comes to *alloca* also. But there are a few differences in the array access patterns between function argument and allocation in the function body. In the LLVM, the former will be compiled into a pointer of array, and the latter is still in the same type. Hence, we have to separate these 2 patterns while lowering memref dialect.

```
1  module attributes {...} {
2    llvm.func @spmm(%arg0: !llvm.ptr<f64>, %arg1: !llvm.ptr
       <f64>, %arg2: !llvm.ptr<f64>) {
3      %0 = llvm.mlir.undef : !llvm.struct<(ptr<f64>, ptr<
       f64>, i64, array<2 x i64>, array<2 x i64>)>
4      %1 = llvm.insertvalue %arg0, %0[0] : !llvm.struct<(
       ptr<f64>, ptr<f64>, i64, array<2 x i64>, array<2 x
       i64>)>
5      ...
6      %12 = llvm.insertvalue %11, %10[4, 1] : !llvm.struct
       <(ptr<f64>, ptr<f64>, i64, array<2 x i64>, array<2 x
        i64>)>
7      %13 = builtin.unrealized_conversion_cast %12 : !
       llvm.struct<(ptr<f64>, ptr<f64>, i64, array<2 x i64
       >, array<2 x i64>)> to memref<3x4xf64>
8      %42 = builtin.unrealized_conversion_cast %13 : memref
       <3x4xf64> to !llvm.struct<(ptr<f64>, ptr<f64>, i64,
       array<2 x i64>, array<2 x i64>)>
9      ...
10     %72 = llvm.extractvalue %42[1] : !llvm.struct<(ptr<
       f64>, ptr<f64>, i64, array<2 x i64>, array<2 x i64>)
       >
11     ...
12     %76 = llvm.getelementptr %72[%75] : (!llvm.ptr<f64>,
       i64) -> !llvm.ptr<f64>
13     %77 = llvm.load %76 : !llvm.ptr<f64>
14     ...
15     llvm.return
16   }
17 }
```

Listing 6: In the current MLIR lowering process, the memref type in function arguments will be transformed into bare pointers of the llvm type when using the *use-bare-ptr-memref-call-conv* flag. As illustrated in Line 2, the arguments are converted to pointers of type f64. However, the access patterns are not compatible with the older version of LLVM IR, as shown in Lines 10 and 12.

After applying these changes, we are able to lower all dialects into the llvm dialect and subsequently translate it into LLVM IR. Listing 7 presents the resulting MLIR code incorporating all the modifications made in our adaptor. In Lines 5-7, we create three arrays using the llvm.alloca operation to store the sparse information. Lines 9-11 obtain the pointers to these three arrays for subsequent sparse functions and load/store operations. Lines 12-14 invoke the three sparse functions to retrieve the information about the sparse matrix and store it in three arrays that we have just allocated.

```
1  module attributes {...} {
2    llvm.func @spmm(%arg0: !llvm.ptr<array<4 x f64>>, %arg1
       : !llvm.ptr<array<2 x f64>>, %arg2: !llvm.ptr<array
       <2 x f64>>) {
3      ...
4      %c1 = llvm.mlir.constant(1 : i32) : i32
5      %5 = llvm.alloca %c1 x !llvm.array<4 x i64> : (i32)
        -> !llvm.ptr<array<4 x i64>>
6      %6 = llvm.alloca %c1 x !llvm.array<12 x i64> : (i32)
        -> !llvm.ptr<array<12 x i64>>
7      %7 = llvm.alloca %c1 x !llvm.array<12 x f64> : (i32)
        -> !llvm.ptr<array<12 x f64>>
8      %c0 = llvm.mlir.constant(0 : i32) : i32
9      %ptr5 = llvm.getelementptr %5[%c0, %c0] : (!llvm.ptr<
       array<4 x i64>>, i32, i32) -> !llvm.ptr<i64>
10     %ptr6 = llvm.getelementptr %6[%c0, %c0] : (!llvm.ptr<
       array<12 x i64>>, i32, i32) -> !llvm.ptr<i64>
11     %ptr7 = llvm.getelementptr %7[%c0, %c0] : (!llvm.ptr<
       array<12 x f64>>, i32, i32) -> !llvm.ptr<f64>
12     llvm.call @sparsePointers(%arg0, %ptr5) : (!llvm.ptr<
       array<4 x f64>>, !llvm.ptr<i64>) -> ()
13     llvm.call @sparseIndices(%arg0, %ptr6) : ...
14     llvm.call @sparseValues(%arg0, %ptr7) : ...
15     ...
16     %gep0 = llvm.getelementptr %ptr5[%8] : ...
17     %11 = llvm.load %gep0 : !llvm.ptr<i64>
18     %13 = llvm.add %8, %0  : i64
```

```
19      ...
20      %14 = llvm.getelementptr %ptr5[%13] : ...
21      ...
22      %gep1 = llvm.getelementptr %ptr6[%17] : ...
23      %20 = llvm.load %gep1 : !llvm.ptr<i64>
24      ...
25      %gep2 = llvm.getelementptr %ptr7[%17] : ...
26      %22 = llvm.load %gep2 : !llvm.ptr<f64>
27      ...
28      %gep3 = llvm.getelementptr %arg2[%8, %23] : (!
         llvm.ptr<array<2 x f64>>, i64, i64) -> !llvm.ptr<f64
         >
29      %26 = llvm.load %gep3 : !llvm.ptr<f64>
30      %gep4 = llvm.getelementptr %arg1[%20, %23] : (!
         llvm.ptr<array<2 x f64>>, i64, i64) -> !llvm.ptr<f64
         >
31      %27 = llvm.load %gep4 : !llvm.ptr<f64>
32      %28 = llvm.fmul %22, %27  : f64
33      %29 = llvm.fadd %26, %28  : f64
34      llvm.store %29, %gep3 : !llvm.ptr<f64>
35      %30 = llvm.add %23, %0  : i64
36      ...
37      llvm.return
38    }
39  }
```

Listing 7: The final result of MLIR in `llvm` dialect. Lines 5-7 is allocating the sparse arrays. Lines 9-11 are extracting these 3 arrays into pointer of arrays for the following functions and load/store operations.

4.2.3 IR-adaptor (MHAL). The remaining steps involve addressing the syntax differences in LLVM IR for the older version of LLVM and adding the necessary pragmas [11], which are represented in attributes in LLVM, and finally getting the HLS readable IR. MLIR uses SSA-form to store registers and basic block labels. It is started from `%0, %1` and so on. This will result in the same naming strategy while translating MLIR to LLVM IR, and this is not the correct syntax for the older version of LLVM, on which HLS tools are based. Pragmas are an important part of direct optimization and resource arrangements. Top function is a necessary pragma to target which function is the top module to be synthesized, hence we add in the final LLVM IR to complete the whole synthesis flow.

5 EXPERIMENTAL RESULTS

To validate the effectiveness of our adaptor, we conducted experiments using Xilinx Vitis HLS as the platform and evaluated the latency and utilization based on the synthesis results. Table 1 provides an overview of the experimental environment. All experiments were performed on Ubuntu 20.04. We utilized Xilinx Vitis HLS version 2021.2, and our adaptor was based on LLVM release/15.x. The experiment flow, benchmarks, and results will be described in the subsequent subsections.

Table 1: Environments of our experiments.

Application	Version
Ubuntu	20.04
Xilinx Vitis HLS	2021.2
LLVM	release/15.x

5.1 Experimental Flow

To evaluate the speed up between original tensor computations (dense) and sparse tensor computations, which are using MLIR compilation flow with our adaptor, we prepare two functions accessing the same data. Figure 3 shows the matrix multiplication in **CSR** compressed format. The top of the right-hand side shows the Python script generating the matrix multiplication MLIR with `SparseTensor` attributes. Then, with the lowering passes and our adaptor, we generate the final MLIR in `llvm` dialect and further translate it into LLVM. Since the syntax is not supported for Xilinx Vitis HLS, which is based on LLVM 7, we also fix the syntax of LLVM IR with our adaptor and finally synthesize it into RTL with Xilinx Vitis HLS (virtex7). For the data and the evaluation, we use Fasttext [7] and MNIST as datasets and evaluate the latency and resource utilization with the report of Xilinx Vitis HLS.

5.2 Test Suites for Sparse Matrix Compression

This experiment focuses on two key scenarios: General Matrix Vector multiplication (GEMV) operations in the FastText model, and the convolution layer from the MNIST dataset. Both of these cases were manipulated to increase the sparsity of matrices involved in computations, aligning with the focus on sparse matrix compression techniques [10].

5.2.1 FastText GEMV. FastText is an efficient library for learning word representations [2] and sentence classification [8]. The General Matrix-Vector multiplication (GEMV) operation in FastText serves as the first test suite in this experiment. This operation is essential to FastText's functionality, transforming input words into vector representations. FastText employs two primary algorithms for generating word vectors: Skipgram and Continuous Bag of Words (CBOW). In this experiment, the Skipgram model is used, which predicts the context given a specific word, proving beneficial for large datasets with infrequent words.

To increase the sparsity, a threshold was set to prune the pretrained word vectors in FastText, setting values below this threshold to zero. This pruning process transforms the original dense pretrained vectors into sparse matrices. After pruning, the model is compressed using the proposed sparse matrix compression method, aiming to reduce the model size and improve its efficiency. The pretrained FastText model, `cc.en.300.bin`, providing 300-dimensional word vectors trained on English Common Crawl data, is utilized for this experiment.

5.2.2 MNIST Convolution Layer. The second test suite involves the convolutional layer of a Convolutional Neural Network (CNN) trained on the MNIST dataset, a large database of handwritten digits frequently used for training various image processing systems. The convolutional layers in CNNs are computationally intensive. Padding was applied to the input images before the convolution operation in this experiment. Padding, the process of adding extra pixels around the input image, allows the convolution operation to cover the edge pixels of the image more thoroughly, but it also results in higher sparsity when the convolution operation is transformed into an im2col convolution operation.

Im2col is a mechanism that converts the 3D input data into a 2D matrix, thereby transforming the convolution operation into

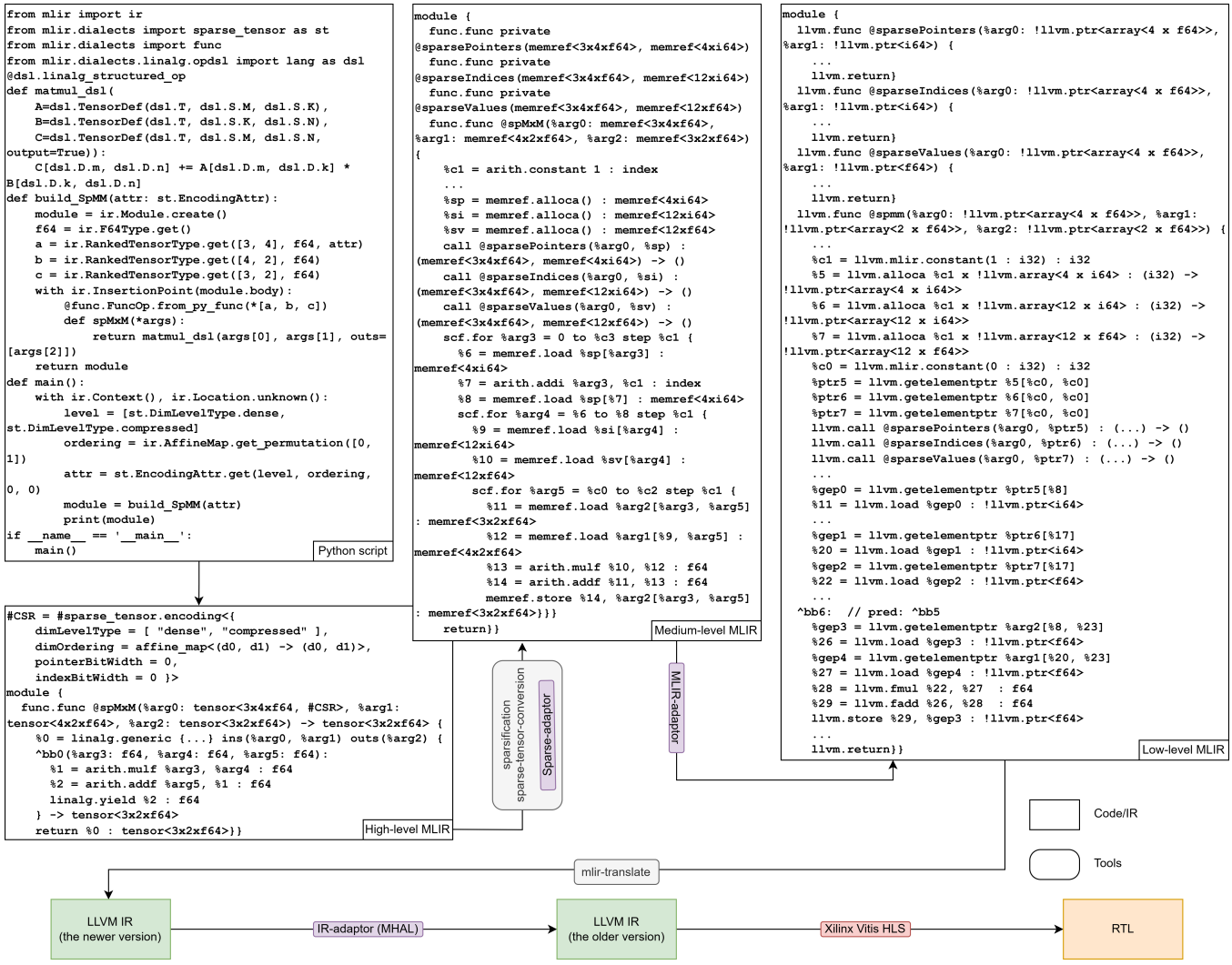


Figure 3: The top of right-hand-side is the Python script generating `C[3,2] = A[3,4] * B[4,2]`. Notice that matrix `A` is with the `SparseTensor` attribute and this figure shows the CSR compress format. The following 3 column contains the high/medium/low level MLIR, which are using standard MLIR lowering flow with our adaptor. All 3 column shows the fixed syntax after our adaptor. The last flow is to translate MLIR into LLVM IR and lower the IR into Xilinx Vitis HLS readable IR with our adaptor, and finally into RTL.

a matrix-matrix multiplication problem. This operation reshapes each receptive field in the input image into a column vector in the output matrix, creating a sparse representation of the original dense convolutional layers. Sparse matrix compression techniques are applied to the weights of these transformed layers, aiming to reduce the computational requirements of the model without significantly impacting its performance.

Both of these test suites present unique challenges and opportunities for sparse matrix compression. By analyzing and comparing the results of these tests, insights into the benefits and limitations of the approach are obtained.

5.3 Evaluation of Different Convolution Layers

In our experiment, we selected one of the convolution layers of the MNIST dataset, which is transformed into a matrix multiplication function using the im2col technique. The final data shape is `25x784` with a sparsity of 75%, and we used a filter of size `32x25`. Thus, we created a function to calculate the matrix multiplication as follows: `C[32,784] = A[32,25] * B[25,784]`, where B represents the sparse matrix from the MNIST dataset. The details of latency and resource utilization are presented in Table 2 and 3, respectively. In the latency part, we only provide the co-simulation result, which represents the estimated cycle count for the given input data. The latency is influenced by the sparsity unless we use `trip count` directives to limit the upper and lower bounds of iterations. The

sparse version achieved a speedup of approximately 3.59 times compared to the dense version. However, in terms of resource utilization, the sparse version exhibits increased flip-flop (FF) usage by a factor of 6 and look-up-table (LUT) usage by a factor of 9. This increase is mainly due to the allocation of three sparse arrays since we maintain the input tensor type and calculate these three arrays at runtime. If the inputs are already compressed, we can generate more specific MLIR and reduce the storage for allocations. This also presents opportunities for further optimizations in future work. For a more detailed analysis of the utilization results, Table 4 provides information on the FF and LUT usage in registers and multiplexers. The sparse version utilizes fewer FFs and LUTs compared to the dense version, as the former performs fewer matrix multiplications. Regarding the experiment on the Fasttext dataset, the synthesis process was not completed due to its large size. We hope to report the results by the workshop presentation.

Table 2: The latency of co-simulation results. Using the MNIST data, the sparse matrix is in 75% sparsity. The sparse one is sped up by 3.59 times.

Input	Latency (cycle)
Dense	627232
Sparse	174775

Table 3: The utilization results are targeting the device on virtex7, which is provided by Xilinx Vitis HLS. The sparse one uses more resources since there have 3 allocations to store sparse information.

Input	*BRAM_18K*	*DSP*	*FF*	*LUT*	*URAM*
Available	1590	1260	728400	364200	0
Dense	0	4	11054	5720	0
Sparse	0	8	59382	48003	0

Table 4: The detail utilization about the multiplexer & registers. The dense one uses more FFs and LUTs since there are much more computations on the zero values, which are in the sparse matrix.

Input	*FF*	*LUT*
Dense	7468	1347
Sparse	4569	595

6 CONCLUSION

Sparse tensor computing significantly reduces the computation latency and data storage. Various scenario uses sparse tensor computing to optimize the program, especially in image-related machine-learning models. Famous models like Pytorch, Tensorflow, etc have already supported sparse tensor computing. MLIR also integrates it and provides a compilation flow to LLVM IR. However, the implementation is not well-done and the lowering feature about the multi-dimension array is not suitable for the HLS tools. We propose an adaptor to complete the MLIR compilation flow and lower the LLVM IR version. Without the gap of unimplemented features and unsupported syntaxes, we can easily integrate various front-end and sparse tensor optimization into HLS to do the synthesis and design more powerful hardware. The experiment result shows that with sparse tensor computing the latency can be largely reduced by about 3.59 times. If we can further utilize the directive optimization with HLS pragmas [6], there will be more powerful RTL to be generated. Ongoing research work also includes experiments with the Fasttext work.

REFERENCES

[1] Aart Bik, Penporn Koanantakool, Tatiana Shpeisman, Nicolas Vasilache, Bixia Zheng, and Fredrik Kjolstad. 2022. Compiler support for sparse tensor computations in MLIR. *ACM Transactions on Architecture and Code Optimization (TACO)* 19, 4 (2022), 1–25.

[2] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2016. Enriching Word Vectors with Subword Information. *arXiv preprint arXiv:1607.04606* (2016).

[3] Rong-Guey Chang, Tyng-Ruey Chuang, and Jenq Kuen Lee. 2004. Support and optimization for parallel sparse programs with array intrinsics of Fortran 90. *Parallel Comput.* 30, 4 (2004), 527–550.

[4] Jonathan Frankle and Michael Carbin. 2018. The lottery ticket hypothesis: Finding sparse, trainable neural networks. *arXiv preprint arXiv:1803.03635* (2018).

[5] Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149* (2015).

[6] Mohammad Hosseinabady and Jose Luis Nunez-Yanez. 2019. A streaming dataflow engine for sparse matrix-vector multiplication using high-level synthesis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 6 (2019), 1272–1285.

[7] Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Hérve Jégou, and Tomas Mikolov. 2016. Fasttext. zip: Compressing text classification models. *arXiv preprint arXiv:1612.03651* (2016).

[8] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2016. Bag of Tricks for Efficient Text Classification. *arXiv preprint arXiv:1607.01759* (2016).

[9] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. https://doi.org/10.1109/CGO51591.2021.9370308

[10] Chao-Lin Lee, Chen-Ting Chao, Wei-Hsu Chu, Ming-Yu Hung, and Jenq-Kuen Lee. 2022. Accelerating AI Applications with Sparse Matrix Compression in Halide. *Journal of Signal Processing Systems* (2022), 1–14.

[11] Geng-Ming Liang, Chuan-Yue Yuan, Meng-Shiun Yuan, Tai-Liang Chen, Kuan-Hsun Chen, and Jenq-Kuen Lee. 2022. The Support of MLIR HLS Adaptor for LLVM IR. In *Workshop Proceedings of the 51st International Conference on Parallel Processing*. 1–8.

[12] Hui-Hsin Liao, Chao-Lin Lee, Jenq-Kuen Lee, Wei-Chih Lai, Ming-Yu Hung, and Chung-Wen Huang. 2021. Support convolution of CNN with compression sparse matrix multiplication flow in TVM. In *50th International Conference on Parallel Processing Workshop*. 1–7.